



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №5
із дисципліни «Технології розроблення програмного забезпечення»
Тема: «Патерни проектування»
Тема проекту: «Музичний програвач»
GitHub: <https://github.com/martyshenkoo/MusicPlayerWeb>

Виконала:
студентка групи ІА-31:
Мартишенко І.С.

Перевірив:
Мягкий М.Ю.

Київ- 2025

Тема: Патерни проектування

Мета: Вивчити структуру шаблонів «Adapter», «Builder», «Command», «Chain of responsibility», «Prototype» та навчитися застосовувати їх в реалізації програмної системи

Музичний програвач (iterator, command, memento, facade, visitor, client-server)

Музичний програвач становить собою програму для програвання музичних файлів або відтворення потокової музики з можливістю створення, запам'ятовування і редагування списків програвання, перемішування/повторення (shuffle/repeat), розпізнавання різних аудіо-форматів, еквалайзер.

Завдання

- Ознайомитись з короткими теоретичними відомостями.
- Реалізувати частину функціоналу робочої програми у вигляді класів та їхньої взаємодії для досягнення конкретних функціональних можливостей.
- Реалізувати один з розглянутих шаблонів за обраною темою.
- Реалізувати не менше 3-х класів відповідно до обраної теми.
- Підготувати звіт щодо виконання лабораторної роботи. Поданий звіт повинен містити: діаграму класів, яка представляє використання шаблону в реалізації системи, навести фрагменти коду по реалізації цього шаблону.

Теоретичні відомості

Призначення патерну: Шаблон "command" (команда) перетворює звичайний виклик методу в клас. Таким чином дії в системі стають повноправними об'єктами. Це зручно в наступних випадках:

- Коли потрібна розвинена система команд – відомо, що команди будуть добавлятися;
- Коли потрібна гнучка система команд – коли з'являється необхідність додавати командам можливість відміни, логування і інш.;
- Коли потрібна можливість складання ланцюжків команд або виклику команд в певний час.

Об'єкт команда сама по собі не виконує ніяких фактичних дій окрім перенаправлення запиту одержувачеві (тобто команди все ж виконуються

одержувачем), однак ці об'єкти можуть зберігати дані для підтримки додаткових функцій відміни, логування і інш. Наприклад, команда вставки символу може запам'ятовувати символ, і при виклику відміни викликати відповідну функцію витирання символу. Можна також визначити параметр «застосовності» команди (наприклад, на картинці писати не можна) – і використати цей атрибут для засвічування піктограми в меню.

Такий підхід до команд дозволяє побудувати дуже гнучку систему команд, що настроюється. У більшості додатків це буде зайвим (використовується спрощений варіант), проте життєво важливий в додатках з великою кількістю команд (редактори).

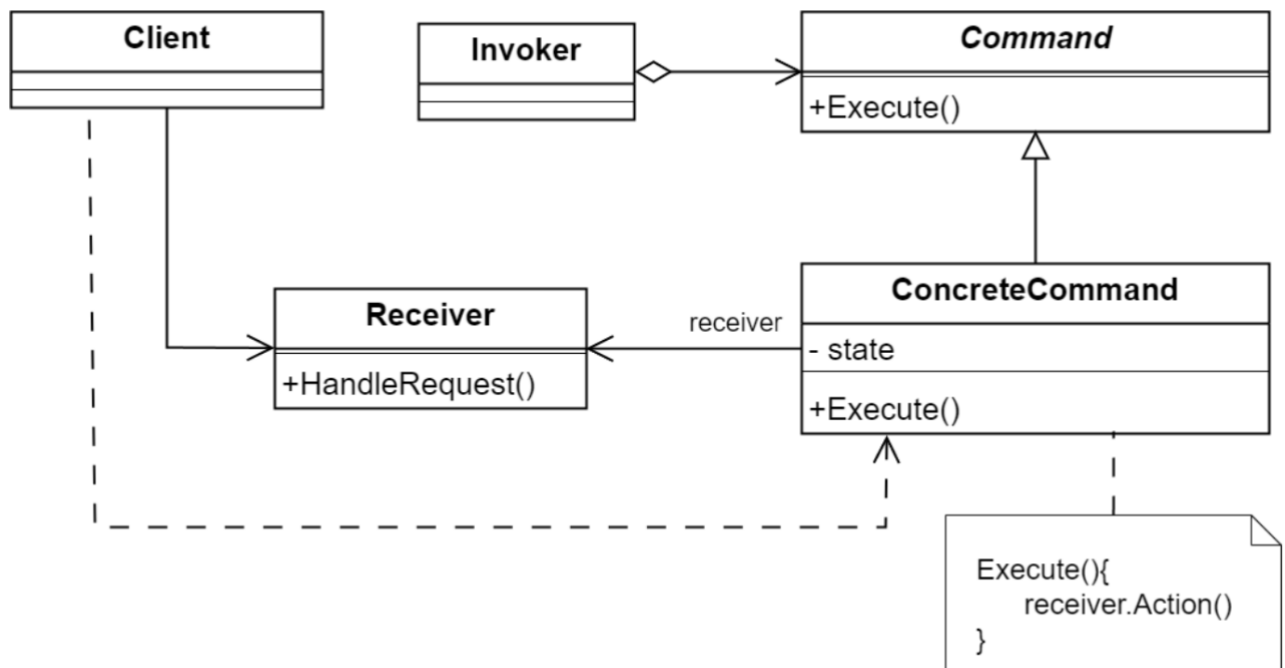


Рисунок 5.3. Структура патерну Команда

Проблема: Ви реалізуєте товстий клієнт який має багатий візуальний інтерфейс: має меню, кнопки і контекстне меню. Кожна дія, яку можна виконати, має три варіанти виконання – через меню, натисканням кнопки та через візуального елементу, але тоді потрібно буде продублювати цей функціонал для меню, кнопки та контекстного меню. Таким чином ми будемо мати дублювання коду і при зміні функціоналу змінювати його в трьох місцях.

Додатковим викликом буде реалізувати автоматизоване тестування такої системи, тому що нам необхідно емулювати натискання кнопок користувачем.

Рішення: Виділення функціоналу який виконується по натисканню на кнопку в окремий клас дозволяє відв'язати візуальну частину від логіки обробки. Таким чином ми будемо мати шар з UI елементами і шар з логікою обробки дій виконаних на UI. Це дозволяє один і той самий об'єкт з шару логіки обробки дій використати для реакцію на натискання кнопки і пункту меню і контекстного меню. Кнопки тепер не знають про конкретні класи реалізації команд, а взаємодіють з об'єктами команд через загальний інтерфейс. Ще одна перевага, що тепер ми можемо достатньо просто реалізувати механізм enable-disable для кнопок та контекстного меню через виклик у об'єкта команди метода `IsEnabled()` або через прив'язку (binding) на поле `IsEnabled` у об'єкта команди.

При такій реалізації також набагато простіше організувати тестування застосунку, тому що ми тепер не потрібно емулювати дії користувача, а достатньо протестувати шар логіки обробки використовуючи модульні тести (unit tests).

Переваги та недоліки:

- + Ініціатор виконання команди не знає деталей реалізації виконавця команди.
 - + Підтримує операції скасування та повторення команд.
 - + Послідовність команд можна логувати і при необхідності виконати цю послідовність ще раз.
 - + Простота розширення за рахунок додавання нових команд без необхідності внесення змін в уже існуючий код (принцип відкритості-закритості).
- контекстне меню. Реалізацію кожної дії можна розмістити в обробнику Патерн Command я обрала тому, що він найкраще підходить для музичного програвача, де кожна дія користувача (Play, Pause, Next, Delete тощо) є окремою командою. Command дозволяє відокремити інтерфейс від логіки, уникнути дублювання коду та легко додавати нові дії без зміни існуючих класів. Крім того, цей патерн підтримує історію дій та скасування операцій, що робить систему більш гнучкою та розширюваною.

Хід роботи

У межах проєкту «Музичний програвач» було реалізовано основну логіку роботи з користувачами та аудіотреками.

Реалізувати частину функціоналу робочої програми у вигляді класів та їхньої взаємодії для досягнення конкретних функціональних можливостей.

```
Controllers > C# PlayerController.cs
1  using Microsoft.AspNetCore.Authorization;
2  using Microsoft.AspNetCore.Mvc;
3  using MusicPlayerWeb.Services;
4  using MusicPlayerWeb.Services.Commands;
5
6  namespace MusicPlayerWeb.Controllers;
7
8  [Authorize]
9  [Route("[controller]/[action]")]
10 public class PlayerController : Controller
11 {
12     private readonly PlayerCommandInvoker _invoker;
13     private readonly PlayerStateService _state;
14
15     public PlayerController(PlayerCommandInvoker invoker, PlayerStateService state)
16     {
17         _invoker = invoker;
18         _state = state;
19     }
20
21     [HttpPost]
22     public IActionResult Play([FromBody] PlayTrackCommandRequest request)
23     {
24         if (request == null || string.IsNullOrEmpty(request.Title) || string.IsNullOrEmpty(request.Url))
25             return BadRequest("Недостатньо даних для програвання треку.");
26
27         var username = User.Identity?.Name ?? "unknown";
28         var command = new PlayTrackCommand(_state, request.Title.Trim(), request.Url);
29         _invoker.Execute(command, username);
30         return Json(new { state = _state.Snapshot, history = _invoker.History });
31     }
32
33     [HttpPost]
34     public IActionResult Pause()
35     {
36         var username = User.Identity?.Name ?? "unknown";
37         var command = new PauseTrackCommand(_state);
38         _invoker.Execute(command, username);
39         return Json(new { state = _state.Snapshot, history = _invoker.History });
40     }
41
42     [HttpPost]
43     public IActionResult Stop()
44     {
45         var username = User.Identity?.Name ?? "unknown";
46         var command = new StopTrackCommand(_state);
47         _invoker.Execute(command, username);
48         return Json(new { state = _state.Snapshot, history = _invoker.History });
49     }
50 }
51
52 public class PlayTrackCommandRequest
53 {
54     public string Title { get; set; } = string.Empty;
55     public string Url { get; set; } = string.Empty;
56 }
57
```

Рис-1. Controllers-PlayerController.cs

```

Services > C# PlayerStateService.cs
1  using MusicPlayerWeb.Models;
2
3  namespace MusicPlayerWeb.Services;
4
5  public class PlayerStateService
6  {
7      private readonly object _sync = new();
8      private PlayerStateSnapshot _snapshot = PlayerStateSnapshot.Empty;
9
10     public PlayerStateSnapshot Snapshot
11     {
12         get
13         {
14             lock (_sync)
15             {
16                 return _snapshot;
17             }
18         }
19     }
20
21     public void Play(string title, string url)
22     {
23         lock (_sync)
24         {
25             _snapshot = PlayerStateSnapshot.Create(title, url, PlayerPlaybackState.Playing);
26         }
27     }
28
29     public void Pause()
30     {
31         lock (_sync)
32         {
33             if (!_snapshot.HasTrack)
34                 return;
35             _snapshot = PlayerStateSnapshot.Create(_snapshot.Title, _snapshot.Url, PlayerPlaybackState.Paused);
36         }
37     }
38
39     public void Stop()
40     {
41         lock (_sync)
42         {
43             _snapshot = PlayerStateSnapshot.Create(null, null, PlayerPlaybackState.Stopped);
44         }
45     }
46 }
47
48

```

Рис-2. Services-PlayerStateService.cs

Реалізувати один з розглянутих шаблонів за обраною темою.

Для реалізації механізму виконання дій у музичному програвачі (таких як додавання трека, видалення трека або перейменування плейліста) було використано патерн Command.

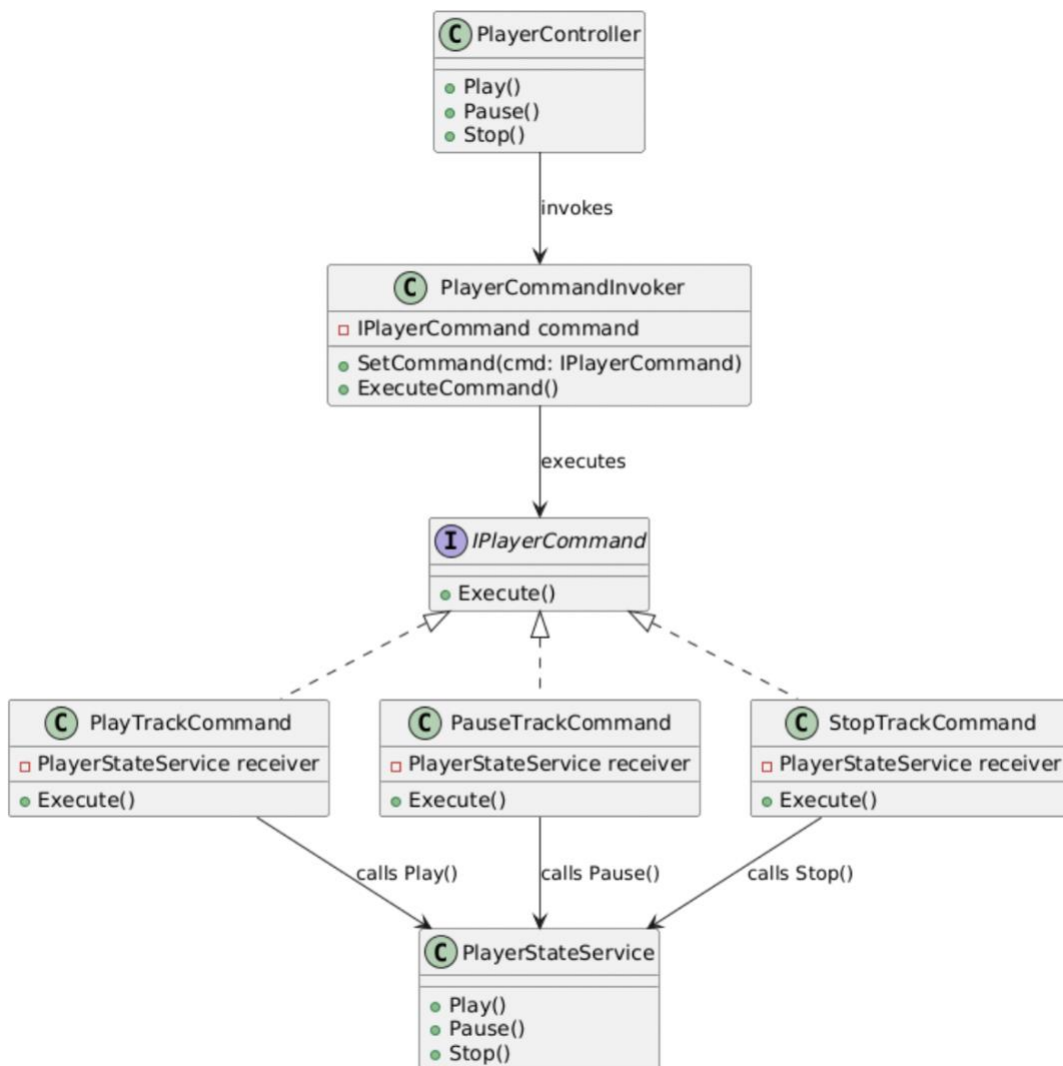


Рис-3. Структура патерну Command

1. IPlayerCommand - інтерфейс команди

Інтерфейс `IPlayerCommand` визначає єдиний метод `Execute()`, який мають реалізовувати всі команди. Це створює спільний контракт для виконання будь-якої дії в програвачі. Завдяки цьому кожна команда поводить себе однаково з точки зору виклику, хоча реалізує різний функціонал.

2. `PlayTrackCommand`, `PauseTrackCommand`, `StopTrackCommand` - конкретні команди

`PlayTrackCommand` викликає метод `Play()` у сервісі стану.

`PauseTrackCommand` викликає `Pause()`.

`StopTrackCommand` викликає `Stop()`.

Кожна з цих команд містить посилання на Receiver і делегує йому фактичне виконання дії. Усі команди реалізують інтерфейс IPlayerCommand.

3. PlayerStateService - одержувач (Receiver)

Клас PlayerStateService є виконавцем команд. Він містить реальну логіку, пов'язану з відтворенням, паузою або зупинкою треку. Команди не містять власної логіки - вони лише викликають відповідні методи цього сервісу.

4. PlayerCommandInvoker - виконавець команд (Invoker)

Invoker (PlayerCommandInvoker) отримує команду через метод SetCommand() та виконує її, викликаючи ExecuteCommand(). Invoker не знає, яку саме операцію буде виконано - він працює лише з інтерфейсом IPlayerCommand.

5. PlayerController - клієнт (Client)

PlayerController виступає у ролі клієнта. Він створює конкретні команди та передає їх Invoker'у. Кожна дія користувача з інтерфейсу перетворюється на об'єкт-команду.

```
Services > Commands > C# IPlayerCommand.cs
1  namespace MusicPlayerWeb.Services.Commands;
2
3  public interface IPlayerCommand
4  {
5      string Name { get; }
6      void Execute();
7  }
8  |
```

Рис-3. Services-Commands-IPlayerCommand.cs


```

Services > Commands > C# PlayTrackCommand.cs
1  namespace MusicPlayerWeb.Services.Commands;
2
3  public class PlayTrackCommand : IPlayerCommand
4  {
5      private readonly PlayerStateService _playerState;
6      private readonly string _title;
7      private readonly string _url;
8
9      public string Name => "Play";
10
11     public PlayTrackCommand(PlayerStateService playerState, string title, string url)
12     {
13         _playerState = playerState;
14         _title = title;
15         _url = url;
16     }
17
18     public void Execute()
19     {
20         _playerState.Play(_title, _url);
21     }
22 }
23

```

Рис-4. Services-Commands-PlayTrackCommand.cs

```

Services > Commands > C# PauseTrackCommand.cs
1  namespace MusicPlayerWeb.Services.Commands;
2
3  public class PauseTrackCommand : IPlayerCommand
4  {
5      private readonly PlayerStateService _playerState;
6
7      public string Name => "Pause";
8
9      public PauseTrackCommand(PlayerStateService playerState)
10     {
11         _playerState = playerState;
12     }
13
14     public void Execute()
15     {
16         _playerState.Pause();
17     }
18 }
19

```

Рис-5. Services-Commands-PauseTrackCommand.cs

```
Services > Commands > C# StopTrackCommand.cs
1  namespace MusicPlayerWeb.Services.Commands;
2
3  public class StopTrackCommand : IPlayerCommand
4  {
5      private readonly PlayerStateService _playerState;
6
7      public string Name => "Stop";
8
9      public StopTrackCommand(PlayerStateService playerState)
10     {
11         _playerState = playerState;
12     }
13
14     public void Execute()
15     {
16         _playerState.Stop();
17     }
18 }
```

Рис-6. Services-Commands-StopTrackCommand.cs

Services > Commands > C# PlayerCommandInvoker.cs

```
1  using MusicPlayerWeb.Models;
2
3  namespace MusicPlayerWeb.Services.Commands;
4
5  public class PlayerCommandInvoker
6  {
7      private const int MaxHistory = 20;
8      private readonly List<PlayerCommandHistoryItem> _history = new();
9
10     public IReadOnlyCollection<PlayerCommandHistoryItem> History
11     {
12         get
13         {
14             lock (_history)
15             {
16                 return _history.ToArray();
17             }
18         }
19     }
20
21     public void Execute(IPlayerCommand command, string username)
22     {
23         command.Execute();
24         AddToHistory(command.Name, username);
25     }
26
27     private void AddToHistory(string commandName, string username)
28     {
29         lock (_history)
30         {
31             _history.Add(new PlayerCommandHistoryItem
32             {
33                 CommandName = commandName,
34                 ExecutedBy = username,
35                 ExecutedAtUtc = DateTime.UtcNow
36             });
37
38             if (_history.Count > MaxHistory)
39             {
40                 _history.RemoveRange(0, _history.Count - MaxHistory);
41             }
42         }
43     }
44 }
```

Рис-7. Services-Commands-PlayerCommandInvoker.cs

Реалізувати не менше 3-х класів відповідно до обраної теми.

Services > C# PlayerStateService.cs

```
1  using MusicPlayerWeb.Models;
2
3  namespace MusicPlayerWeb.Services;
4
5  public class PlayerStateService
6  {
7      private readonly object _sync = new();
8      private PlayerStateSnapshot _snapshot = PlayerStateSnapshot.Empty;
9
10     public PlayerStateSnapshot Snapshot
11     {
12         get
13         {
14             lock (_sync)
15             {
16                 return _snapshot;
17             }
18         }
19     }
20
21     public void Play(string title, string url)
22     {
23         lock (_sync)
24         {
25             _snapshot = PlayerStateSnapshot.Create(title, url, PlayerPlaybackState.Playing);
26         }
27     }
28
29     public void Pause()
30     {
31         lock (_sync)
32         {
33             if (!_snapshot.HasTrack)
34                 return;
35
36             _snapshot = PlayerStateSnapshot.Create(_snapshot.Title, _snapshot.Url, PlayerPlaybackState.Paused);
37         }
38     }
39
40     public void Stop()
41     {
42         lock (_sync)
43         {
44             _snapshot = PlayerStateSnapshot.Create(null, null, PlayerPlaybackState.Stopped);
45         }
46     }
47 }
```

Рис-8. Services – PlayerStateService.cs

```

Models > C# PlayerCommandHistoryItem.cs
1  namespace MusicPlayerWeb.Models;
2
3  public class PlayerCommandHistoryItem
4  {
5      public required string CommandName { get; init; }
6      public DateTime ExecutedAtUtc { get; init; } = DateTime.UtcNow;
7      public string ExecutedBy { get; init; } = "system";
8  }
9

```

Рис-9. Models – PlayerCommandHistoryItem.cs

```

Models > C# PlayerStateSnapshot.cs
1  namespace MusicPlayerWeb.Models;
2
3  public class PlayerStateSnapshot
4  {
5      public string? Title { get; init; }
6      public string? Url { get; init; }
7      public PlayerPlaybackState State { get; init; }
8      public DateTime UpdatedAtUtc { get; init; }
9
10     public bool HasTrack => !string.IsNullOrEmpty(Title) && !string.IsNullOrEmpty(Url);
11
12     public static PlayerStateSnapshot Create(string? title, string? url, PlayerPlaybackState state)
13     {
14         return new PlayerStateSnapshot
15         {
16             Title = title,
17             Url = url,
18             State = state,
19             UpdatedAtUtc = DateTime.UtcNow
20         };
21     }
22
23     public static PlayerStateSnapshot Empty => Create(null, null, PlayerPlaybackState.Stopped);
24 }
25

```

Рис-10. Models – PlayerStateSnapshot.cs

Висновок:

У ході виконання лабораторної роботи було опрацьовано сучасні патерни проєктування, серед яких Adapter, Builder, Command, Chain of Responsibility та Prototype. Детальне ознайомлення з їхньою структурою та принципами роботи дозволило сформуванати глибше розуміння того, як шаблони допомагають підвищувати гнучкість, масштабованість та підтримуваність програмних систем. На прикладі веб-застосунку «Музичний програвач» було реалізовано патерн Command, який застосовано для організації виконання операцій у вигляді окремих команд. Такий підхід дозволив інкапсулювати логіку додавання, видалення та редагування елементів

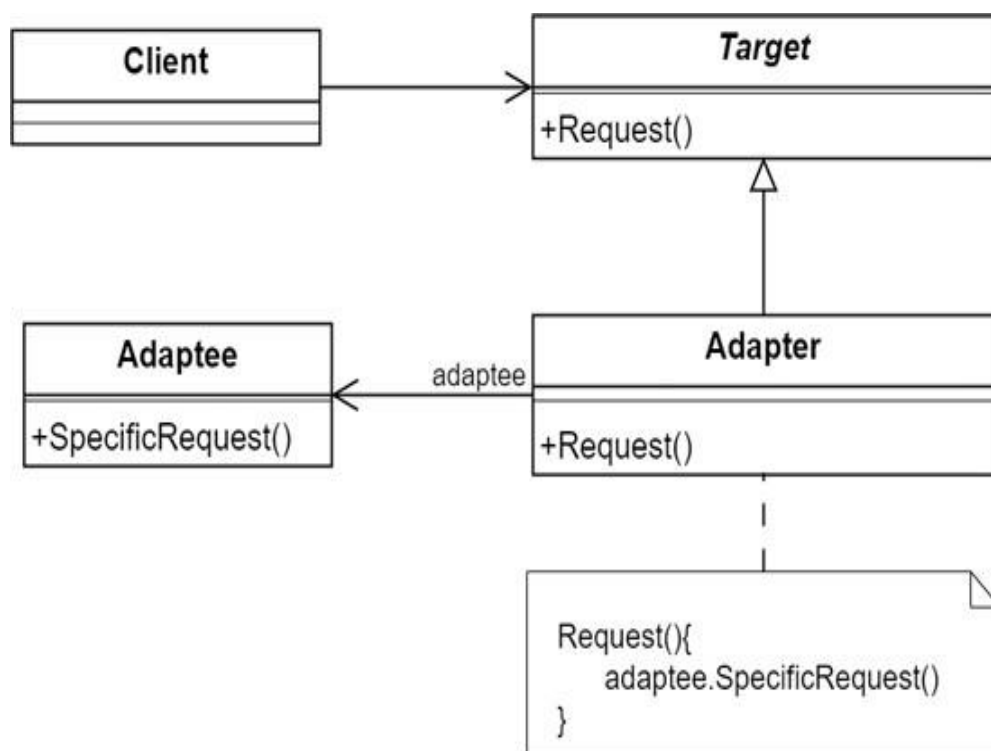
у структурованих класах, відокремити її від контролерів та забезпечити можливість подальшого розширення функціоналу без зміни вже написаного коду. Практична реалізація патерну Command дала змогу переконатися в тому, що шаблони проєктування є важливим інструментом створення якісних програмних продуктів, оскільки вони сприяють кращій організації архітектури та спрощують підтримку проєкту. Виконання роботи закріпило теоретичні знання та показало, що використання патернів значно покращує структуру програмних рішень і робить їх більш гнучкими до майбутніх змін.

Відповіді на контрольні запитання:

1. Яке призначення шаблону «Адаптер»?

Забезпечує сумісність між двома інтерфейсами, які спочатку не можуть працювати разом. Адаптер «обгортає» існуючий клас і перетворює його інтерфейс у той, який очікує клієнт.

2. Нарисуйте структуру шаблону «Адаптер».



3. Які класи входять в шаблон «Адаптер», та яка між ними взаємодія?

- Client - використовує об'єкт через інтерфейс Target.
- Target - описує інтерфейс, який очікує клієнт.
- Adaptee - існуючий клас із несумісним інтерфейсом.
- Adapter - реалізує Target, але всередині викликає методи Adaptee.

4. Яка різниця між реалізацією «Адаптера» на рівні об'єктів та на рівні класів?

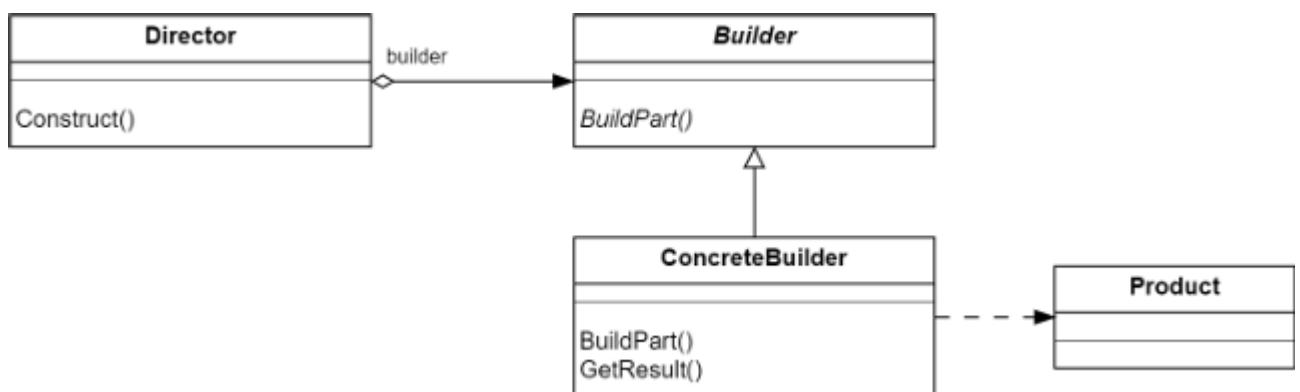
- Об'єктний адаптер - використовує композицію (всередині містить екземпляр Adaptee).
- Класовий адаптер - використовує спадкування (наслідує Adaptee і реалізує Target).

Об'єктний підхід гнучкіший, бо дозволяє адаптувати кілька класів одночасно.

5. Яке призначення шаблону «Будівельник»?

Відокремлює процес створення складного об'єкта від його представлення, дозволяючи будувати різні варіанти об'єкта з однієї і тієї ж послідовності кроків.

6. Нарисуйте структуру шаблону «Будівельник».



7. Які класи входять в шаблон «Будівельник», та яка між ними взаємодія?

- Builder - інтерфейс, що описує кроки побудови об'єкта.
- ConcreteBuilder - конкретна реалізація, що створює частини продукту.
- Director - керує послідовністю викликів методів будівельника.
- Product - кінцевий складний об'єкт.

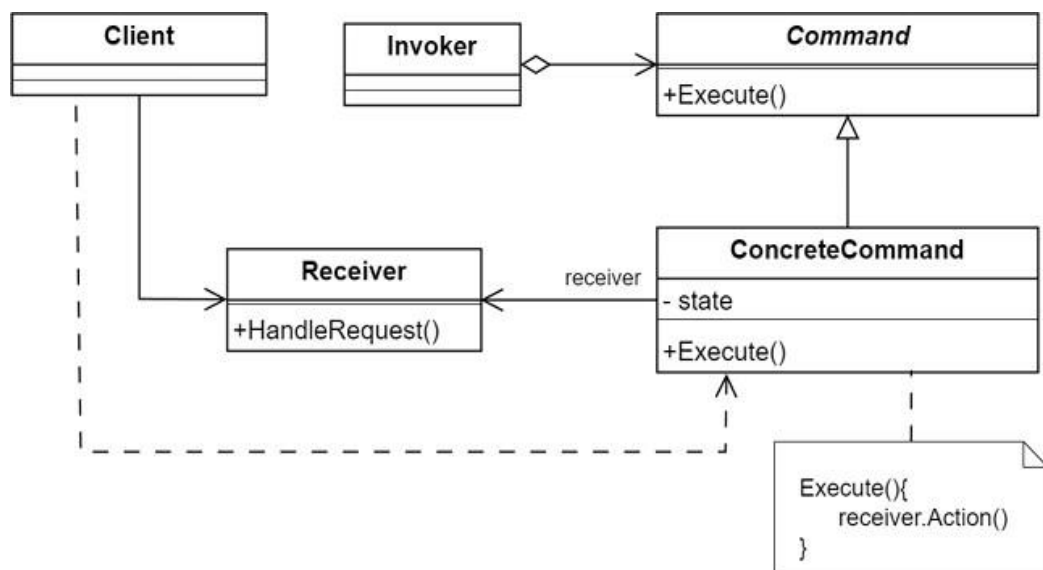
8. У яких випадках варто застосовувати шаблон «Будівельник»?

Коли об'єкт потрібно створювати поступово або в кілька кроків, і є багато варіантів конфігурації цього об'єкта.

9. Яке призначення шаблону «Команда»?

Інкапсулює запит (дію) як об'єкт, що дозволяє параметризувати об'єкти діями, ставити їх у чергу або скасовувати операції.

10. Нарисуйте структуру шаблону «Команда».



11. Які класи входять в шаблон «Команда», та яка між ними взаємодія?

- Command - інтерфейс із методом Execute().
- ConcreteCommand - реалізує команду, викликаючи метод у Receiver.
- Receiver - виконує реальну роботу.
- Invoker - зберігає команду і викликає її виконання.
- Client - створює команду й призначає її виконавцю.

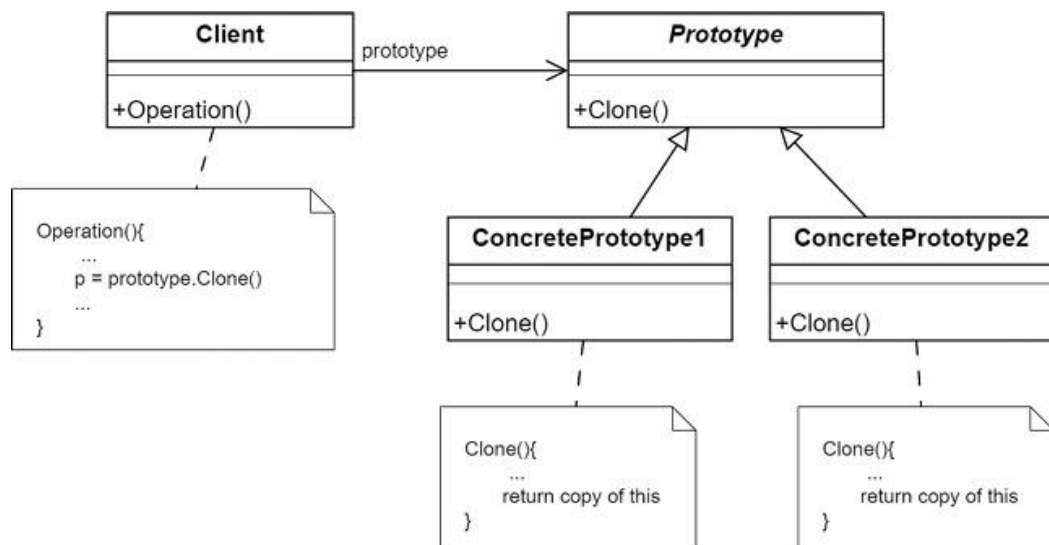
12. Розкажіть як працює шаблон «Команда».

Клієнт створює команду і задає отримувача. Коли Invoker викликає метод Execute(), команда звертається до Receiver, який виконує необхідну дію. Це дозволяє відділити відправника запиту від його виконання.

13. Яке призначення шаблону «Прототип»?

Створює нові об'єкти шляхом копіювання вже існуючих екземплярів (прототипів), замість створення через конструктор.

14. Нарисуйте структуру шаблону «Прототип».



15. Які класи входять в шаблон «Прототип», та яка між ними взаємодія?

- **Prototype** — оголошує метод `Clone()`.
- **ConcretePrototype** — реалізує клонування самого себе.
- **Client** — створює нові об'єкти через метод `Clone()`.

16. Які можна привести приклади використання шаблону «Ланцюжок відповідальності»?

- Обробка запитів у системах підтримки (спочатку оператор, потім менеджер, потім директор).
- Фільтрація запитів у веб-додатках (middleware).
- Обробка подій у GUI (коли подія передається вгору по ієрархії елементів інтерфейсу).